SQL interview Questions

1. What is the difference between DELETE and TRUNCATE commands in SQL?

Answer:

- **DELETE:** Deletes specific rows based on a condition. It logs each deletion, can be rolled back, and can include a WHERE clause.
- **TRUNCATE:** Deletes all rows from a table. It is faster as it deallocates the data pages, does not log individual row deletions, and cannot be rolled back.

---

2. How does a JOIN operation work in SQL, and what are its types?

Answer:
JOIN combines rows from two or more tables based on a related column.

- **INNER JOIN:** Returns rows with matching values in both tables.
- **LEFT JOIN (or LEFT OUTER JOIN):** Returns all rows from the left table and matched rows from the right.
- **RIGHT JOIN (or RIGHT OUTER JOIN):** Returns all rows from the right table and matched rows from the left.
- **FULL OUTER JOIN:** Returns all rows when there is a match in either table.
- **CROSS JOIN:** Produces the Cartesian product of two tables.

---

3. What is the difference between the WHERE and HAVING clauses?

Answer:

- **WHERE:** Filters rows before grouping. It applies to individual rows.
- **HAVING:** Filters groups after aggregation. It applies to grouped data.

---

4. What are Primary Keys and Foreign Keys in SQL?

Answer:

- **Primary Key:** A column or set of columns that uniquely identifies each row in a table. It cannot have NULL values.
- **Foreign Key:** A column in one table that establishes a relationship with the Primary Key in another table.

---

5. How does UNION differ from UNION ALL?

Answer:

- **UNION:** Combines results from two queries and removes duplicate rows.
- **UNION ALL:** Combines results from two queries without removing duplicates.

---

6. How does the GROUP BY clause work in SQL?

Answer:
The **GROUP BY** clause groups rows with the same values in specified columns. It is often used with aggregate functions (e.g., COUNT, SUM, AVG) to perform calculations for each group.

---

7. What are the differences between INNER JOIN and OUTER JOIN?

Answer:

- **INNER JOIN:** Returns only rows with matching values in both tables.
- **OUTER JOIN:** Includes rows that do not have matching values in one table (LEFT, RIGHT, or FULL).

---

8. What is a Subquery, and how is it used in SQL?

Answer:
A **Subquery** is a query nested within another query. It can be used:

- In SELECT statements for calculated columns.
- In WHERE clauses for filtering results.
- In FROM clauses to create derived tables.

---

9. What is the difference between RANK(), DENSE_RANK(), and ROW_NUMBER()?

Answer:

- **RANK():** Assigns ranks with gaps for ties.
- **DENSE_RANK():** Assigns ranks without gaps for ties.
- **ROW_NUMBER():** Assigns a unique number to each row, regardless of ties.

---

10. What is indexing in SQL, and what are its types?

Answer:
Indexing improves query performance by creating a data structure for quick lookups.

- **Clustered Index:** Arranges table data physically in the index order.

- **Non-clustered Index:** Maintains a separate structure for indexing, leaving table data unchanged.

---

11. How does the IN clause differ from the EXISTS clause?

Answer:

- **IN:** Checks if a value exists in a list of specified values.
- **EXISTS:** Checks for the existence of rows returned by a subquery.

---

12. What are aggregate functions in SQL? Provide examples.

Answer:
Aggregate functions perform calculations on a set of rows. Examples:

- **COUNT():** Counts rows.
- **SUM():** Calculates the sum of values.
- **AVG():** Computes the average.
- **MIN():** Finds the smallest value.
- **MAX():** Finds the largest value.

---

13. How do you write a query to identify duplicate rows in a table?

Answer:

```
SELECT column1, column2, COUNT(*)
FROM table_name
GROUP BY column1, column2
HAVING COUNT(*) > 1;
```

---

14. Explain the concept of Normalization in SQL.

Answer:
Normalization organizes data to reduce redundancy and improve integrity. It involves dividing data into multiple tables and defining relationships between them. Common forms: 1NF, 2NF, 3NF, BCNF.

---

15. How does the CASE statement work in SQL?

Answer:
The **CASE** statement provides conditional logic in SQL.

```
SELECT column1,
    CASE
      WHEN condition THEN result1
      ELSE result2
    END AS alias_name
FROM table_name;
```

---

16. What is a Self-Join? Can you provide an example?

Answer:
A **Self-Join** joins a table to itself. It is useful for comparing rows within the same table.
Example:

```
SELECT A.employee_id, A.name, B.name AS manager_name
FROM employees A
JOIN employees B
ON A.manager_id = B.employee_id;
```

---

17. What is the difference between correlated and non-correlated subqueries?

Answer:

- **Correlated Subquery:** Depends on the outer query. It executes once for each row in the outer query.
- **Non-correlated Subquery:** Independent of the outer query. It executes only once.

---

18. Can you explain the purpose of window functions in SQL?

Answer:
Window functions perform calculations across rows related to the current row. Examples:

- **RANK()**, **ROW_NUMBER()**, **SUM()**, **AVG()** with OVER() clause.
  Example:

```
SELECT employee_id, salary,
    RANK() OVER (ORDER BY salary DESC) AS rank
FROM employees;
```

---

19. How do CAST and CONVERT differ in SQL?

Answer:

- **CAST:** ANSI-compliant. Converts one data type to another.
- SELECT CAST(column AS datatype) FROM table;
- **CONVERT:** SQL Server-specific. Offers style formatting for certain data types.
- SELECT CONVERT(datatype, column, style) FROM table;

---

20. What are some best practices for optimizing SQL queries?

Answer:

- Use proper indexing.
- Avoid SELECT *.
- Use JOINS instead of subqueries when possible.
- Optimize WHERE clauses with indexed columns.
- Use LIMIT or TOP to restrict returned rows.

---

21. What are constraints in SQL?

Answer:
Constraints ensure data integrity in a table. Common types:

- **NOT NULL:** Ensures a column cannot have NULL values.
- **UNIQUE:** Ensures unique values in a column.
- **PRIMARY KEY:** Uniquely identifies each row.
- **FOREIGN KEY:** Enforces referential integrity.
- **CHECK:** Validates data based on a condition.
- **DEFAULT:** Provides default values for a column.

---

22. How do the LIMIT and TOP clauses work in SQL?

Answer:

- **LIMIT:** Restricts the number of rows returned (MySQL, PostgreSQL).
- SELECT * FROM table_name LIMIT 10;
- **TOP:** Limits rows (SQL Server).
- SELECT TOP 10 * FROM table_name;

---

23. What is the difference between CHAR and VARCHAR data types?

Answer:

- **CHAR:** Fixed-length storage. Padded with spaces if the value is shorter.
- **VARCHAR:** Variable-length storage. Only stores the exact number of characters.

---

24. What is a CTE (Common Table Expression), and how is it used?

Answer:
A **CTE** is a temporary result set used within a query.
Example:

```
WITH CTE_Name AS (
  SELECT column1, column2
  FROM table_name
)
SELECT * FROM CTE_Name WHERE column1 > 10;
```

---

25. How would you retrieve the nth highest salary in SQL?

Answer:
Using **LIMIT**:

```
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT n-1, 1;
```

Using **ROW_NUMBER()**:

```
WITH RankedSalaries AS (
  SELECT salary, ROW_NUMBER() OVER (ORDER BY salary DESC) AS rank
  FROM employees
)
SELECT salary FROM RankedSalaries WHERE rank = n;
```

---

26. How does DISTINCT differ from GROUP BY?

Answer:

- **DISTINCT:** Eliminates duplicate rows in the result set.

- **GROUP BY:** Groups rows and allows aggregate functions to be applied to each group.

---

27. What is the purpose of the COALESCE function in SQL?

Answer:
COALESCE returns the first non-NULL value in a list of expressions.

```
SELECT COALESCE(column1, column2, 'default') FROM table_name;
```

---

28. How do you use the MERGE statement in SQL?

Answer:
MERGE allows inserting, updating, or deleting rows based on a condition.
Example:

```
MERGE INTO target_table AS target
USING source_table AS source
ON target.id = source.id
WHEN MATCHED THEN
  UPDATE SET target.name = source.name
WHEN NOT MATCHED THEN
  INSERT (id, name) VALUES (source.id, source.name);
```

---

29. What are triggers in SQL, and how do they work?

Answer:
Triggers are automatic actions executed in response to database events (INSERT, UPDATE, DELETE).
Example:

```
CREATE TRIGGER trigger_name
AFTER INSERT ON table_name
FOR EACH ROW
BEGIN
  INSERT INTO log_table(action, timestamp) VALUES ('insert', NOW());
END;
```

---

30. Can you explain the difference between a view and a materialized view?

Answer:

- **View:** A virtual table based on a query. It doesn't store data.
- **Materialized View:** Stores query results physically. It improves performance for complex queries.

---

31. How does the DELETE command differ from DROP?

Answer:

- **DELETE:** Removes rows but retains the table structure.
- **DROP:** Deletes the entire table, including its structure and data.

---

32. How do you create a stored procedure in SQL?

Answer:

```
CREATE PROCEDURE procedure_name (parameters)
AS
BEGIN
  SQL statements;
END;
```

---

33. What is the purpose of the NULLIF function in SQL?

Answer:
NULLIF returns NULL if two expressions are equal; otherwise, it returns the first expression.

```
SELECT NULLIF(column1, column2) FROM table_name;
```

---

34. How does SET differ from SELECT in SQL?

Answer:

- **SET:** Assigns a value to a variable. Used for one variable at a time.
- **SELECT:** Assigns values to variables and can fetch multiple rows.

---

35. How can you perform a recursive query in SQL?

Answer:
Using a **CTE**:

```
WITH RecursiveCTE AS (
  SELECT column1 FROM table_name WHERE condition
```

```
  UNION ALL
  SELECT column1 FROM table_name WHERE condition_based_on_RecursiveCTE
)
SELECT * FROM RecursiveCTE;
```

36. What is a clustered index, and how is it different from a non-clustered index?

Answer:

- **Clustered Index:** Sorts and stores the data rows physically in the table based on the index key. A table can have only one clustered index.
- **Non-clustered Index:** Maintains a separate structure for the index, with pointers to the physical data rows. A table can have multiple non-clustered indexes.

---

37. How would you write a query to fetch data in a hierarchical structure?

Answer:
Using a **recursive CTE**:

```
WITH Hierarchy AS (
  SELECT employee_id, manager_id, 1 AS level
  FROM employees
  WHERE manager_id IS NULL
  UNION ALL
  SELECT e.employee_id, e.manager_id, h.level + 1
  FROM employees e
  JOIN Hierarchy h ON e.manager_id = h.employee_id
)
SELECT * FROM Hierarchy;
```

---

38. What is the difference between ISNULL() and IFNULL()?

Answer:

- **ISNULL():** Used in SQL Server to replace NULL with a specified value.
- SELECT ISNULL(column, 'default_value') FROM table;
- **IFNULL():** Used in MySQL to achieve the same purpose.
- SELECT IFNULL(column, 'default_value') FROM table;

---

39. Can you explain the difference between DATE, DATETIME, and TIMESTAMP?

Answer:

- **DATE:** Stores only date values (e.g., YYYY-MM-DD).
- **DATETIME:** Stores both date and time values (e.g., YYYY-MM-DD HH:MM:SS).
- **TIMESTAMP:** Stores date and time as a UTC value. It is affected by time zones.

---

40. How are wildcards used in SQL queries?

Answer:
Wildcards are used in the **LIKE** operator for pattern matching:

- %: Matches zero or more characters.
- SELECT * FROM table WHERE column LIKE 'abc%';
- _: Matches exactly one character.
- SELECT * FROM table WHERE column LIKE 'a_c';

---

41. How can you pivot data in SQL?

Answer:
Using a **CASE** statement or the PIVOT operator:
Example using CASE:

```
SELECT category,
    SUM(CASE WHEN year = '2023' THEN sales ELSE 0 END) AS sales_2023,
    SUM(CASE WHEN year = '2024' THEN sales ELSE 0 END) AS sales_2024
FROM sales_table
GROUP BY category;
```

---

42. What is the difference between the EXCEPT and MINUS operators?

Answer:

- **EXCEPT:** Used in SQL Server and PostgreSQL to return rows from the first query that are not in the second query.
- **MINUS:** Used in Oracle for the same purpose.

---

43. How do you update multiple rows with different values in SQL?

Answer:
Using a CASE statement:

```
UPDATE table_name
SET column_name = CASE
  WHEN condition1 THEN value1
  WHEN condition2 THEN value2
END
WHERE condition_column IN (condition1, condition2);
```

---

44. What is a transaction in SQL, and what are its ACID properties?

Answer:
A transaction is a sequence of operations treated as a single logical unit.
ACID properties:

- **Atomicity:** All operations complete or none do.
- **Consistency:** Ensures the database remains in a valid state.
- **Isolation:** Transactions are executed independently.
- **Durability:** Changes are permanent once committed.

---

45. How do you write a query to calculate the cumulative sum of a column?

Answer:
Using the **WINDOW FUNCTION**:

```
SELECT column_name,
   SUM(column_name) OVER (ORDER BY some_column) AS cumulative_sum
FROM table_name;
```

---

46. What is the purpose of ROLLUP and CUBE in SQL?

Answer:

- **ROLLUP:** Generates subtotals and grand totals for a hierarchy of groups.
- SELECT column1, column2, SUM(column3)
- FROM table_name
- GROUP BY ROLLUP (column1, column2);
- **CUBE:** Generates subtotals for all combinations of columns.

- SELECT column1, column2, SUM(column3)
- FROM table_name
- GROUP BY CUBE (column1, column2);

---

47. How would you write a query to find all employees who joined within the last six months?

Answer:

SELECT *

FROM employees

WHERE join_date >= DATEADD(MONTH, -6, GETDATE());

Follow Ahmed Mohiuddin | LinkedIn